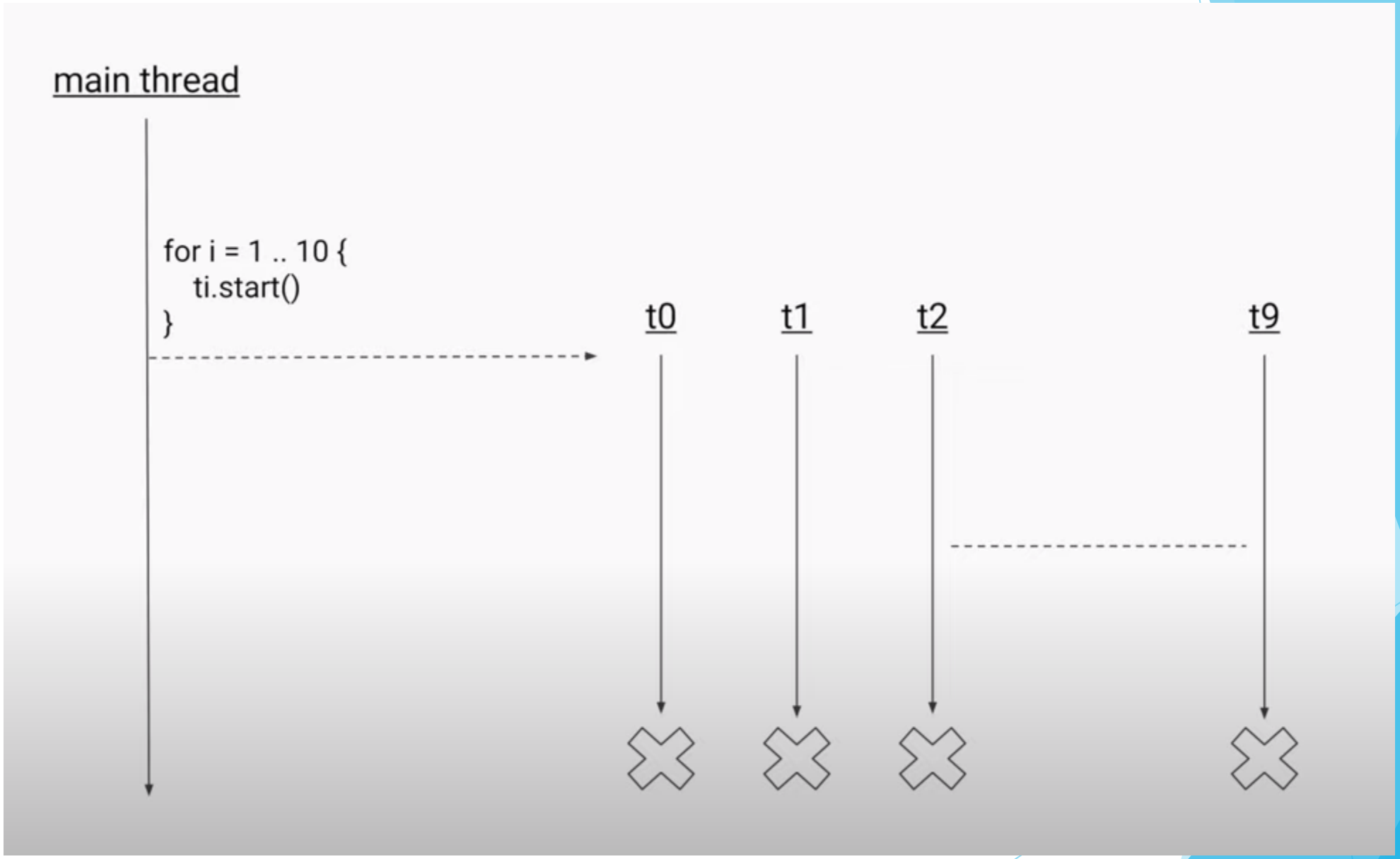
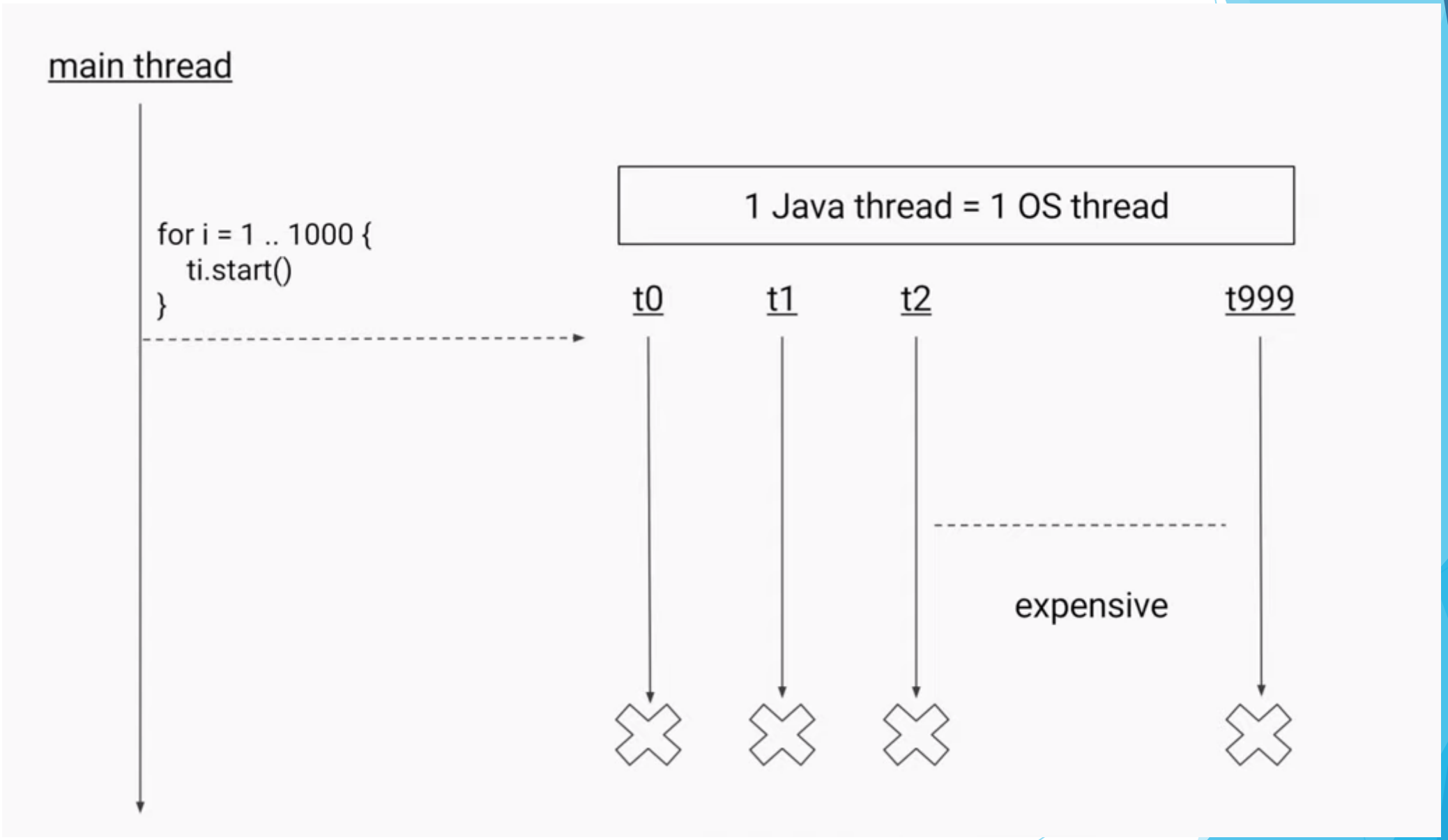
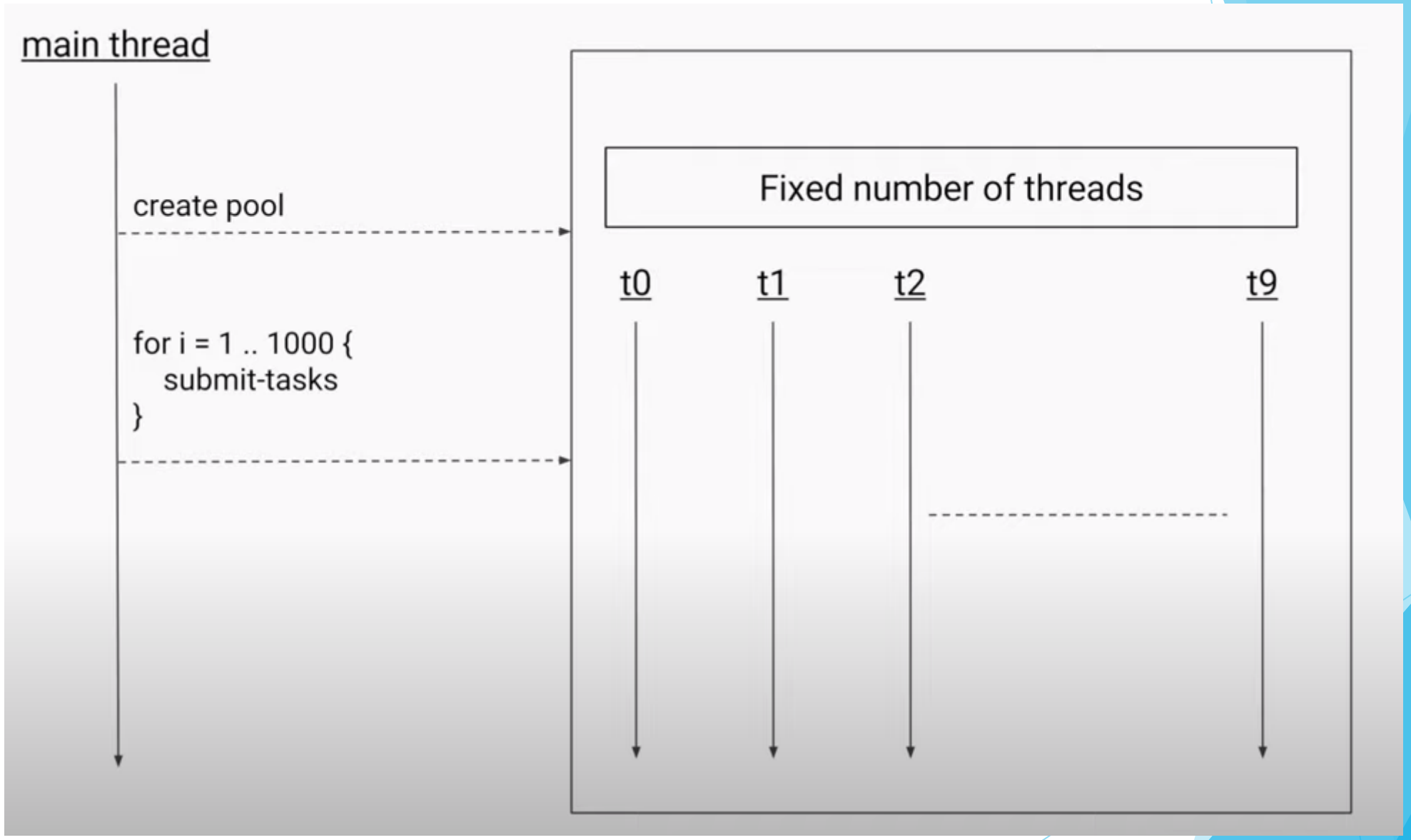
JAVA CONCURRENCY
Advanced Topics

```java
public static void main(String[] args) {

    Thread thread1 = new Thread(new Task());
    thread1.start();
    System.out.println("Thread Name: " + Thread.currentThread().getName());
}

static class Task implements Runnable {
    public void run() {
        System.out.println("Thread Name: " + Thread.currentThread().getName());
    }
}
```

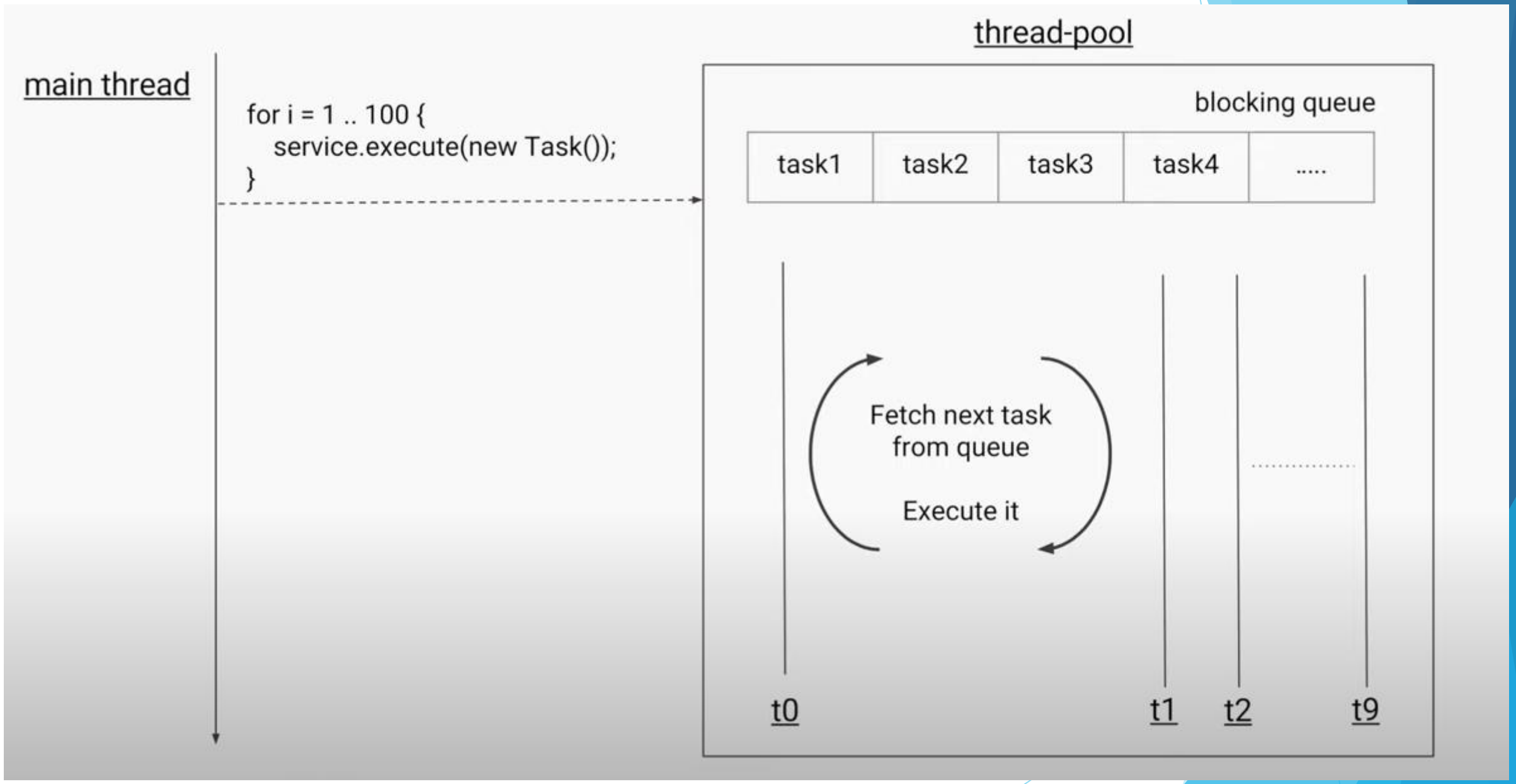
```java
public static void main(String[] args) {

    for (int i = 0; i < 10; i++) {
        Thread thread = new Thread(new Task());
        thread.start();
    }
    System.out.println("Thread Name: " + Thread.currentThread().getName());
}

static class Task implements Runnable {
    public void run() {
        System.out.println("Thread Name: " + Thread.currentThread().getName());
    }
}
```

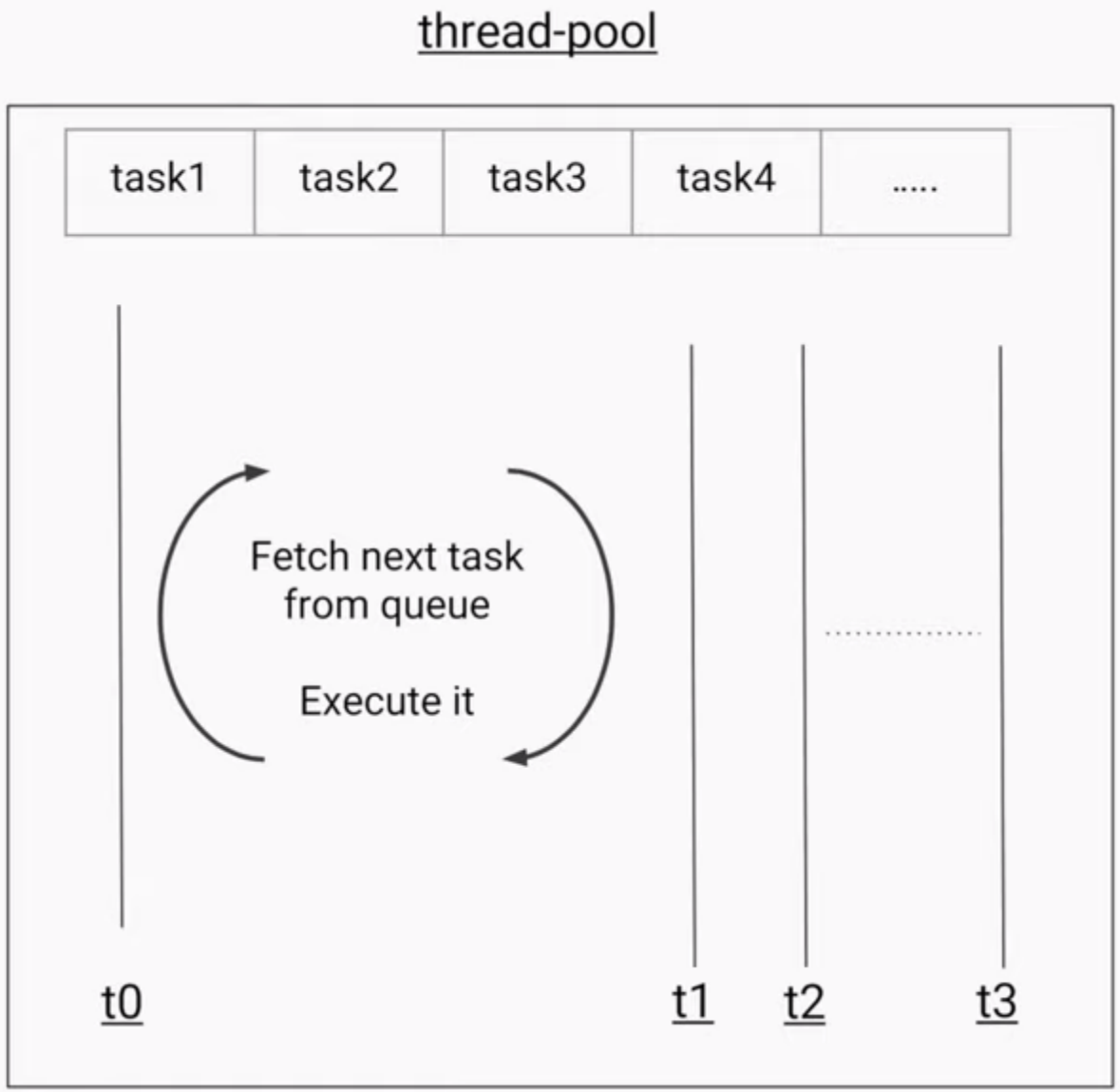

main thread
for i = 1 .. 10 {
ti.start()
}
t0
t1
t2
t9


main thread
for i = 1 .. 1000 {
ti.start()
}
1 Java thread = 1 OS thread
t0
t1
t2
t999
expensive


main thread
create pool
for i = 1 .. 1000 {
submit-tasks
}
Fixed number of threads
t0
t1
t2
t9

```java
public static void main(String[] args) {

    // create the pool
    ExecutorService service = Executors.newFixedThreadPool( nThreads: 10);

    // submit the tasks for execution
    for (int i = 0; i < 100; i++) {
        service.execute(new Task());
    }
    System.out.println("Thread Name: " + Thread.currentThread().getName());
}

static class Task implements Runnable {
    public void run() {
        System.out.println("Thread Name: " + Thread.currentThread().getName());
    }
}
```


thread-pool
main thread
for i = 1 .. 100 {
service.execute(new Task());
}
blocking queue
task1
task2
task3
task4
.....
Fetch next task
from queue
Execute it
t0
t1
t2
t9

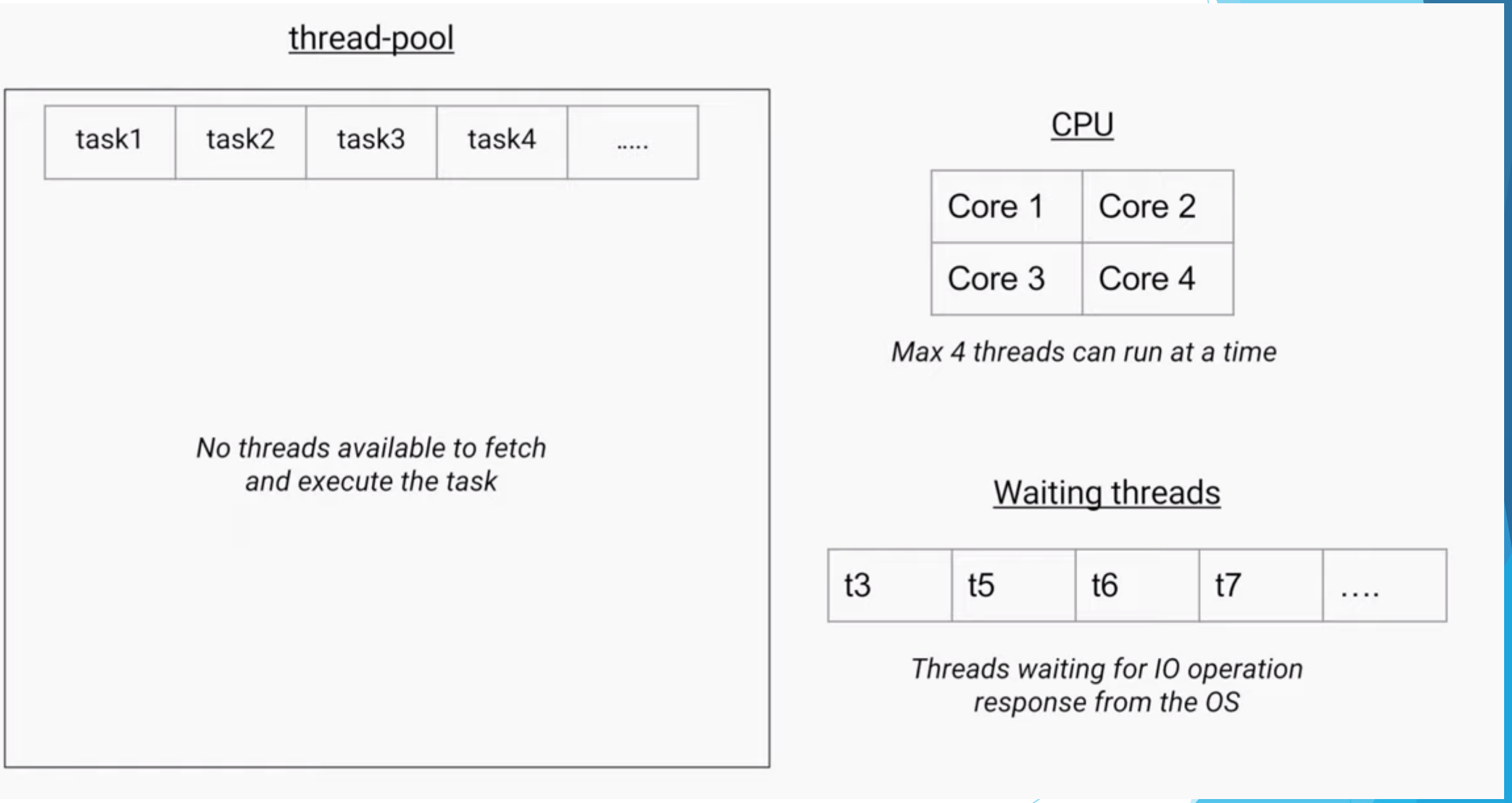


CPU

Core 1	Core 2
Core 3	Core 4

Max 4 threads can run at a time

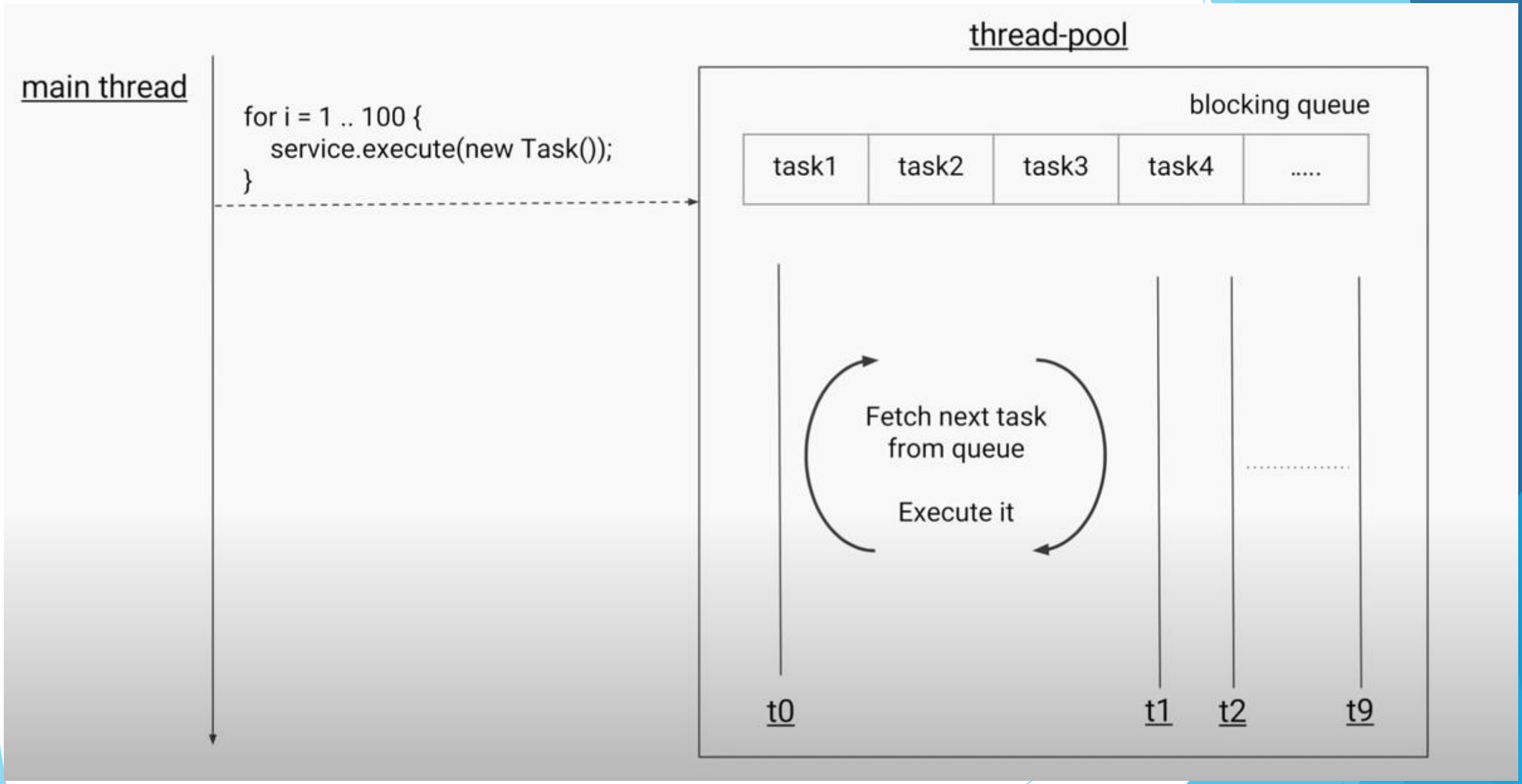
```java
public static void main(String[] args) {

    // get count of available cores
    int coreCount = Runtime.getRuntime().availableProcessors();
    ExecutorService service = Executors.newFixedThreadPool(coreCount);

    // submit the tasks for execution
    for (int i = 0; i < 100; i++) {
        service.execute(new CpuIntensiveTask());
    }
}

static class CpuIntensiveTask implements Runnable {
    public void run() {
        // some CPU intensive operations
    }
}
```

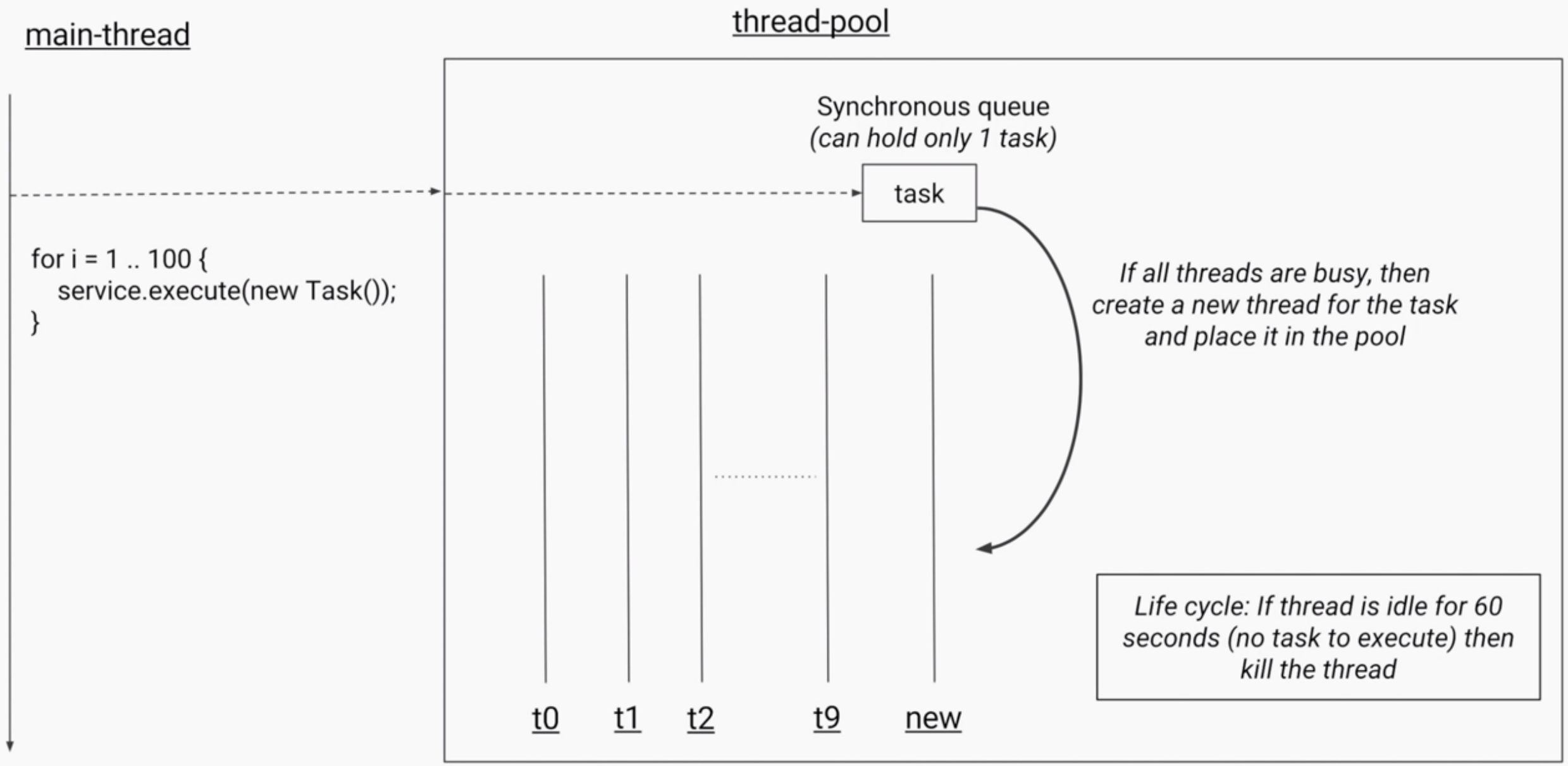

thread-pool
task1
task2
task3
task4
.....
No threads available to fetch and execute the task
CPU
Core 1
Core 2
Core 3
Core 4
Max 4 threads can run at a time
Waiting threads
t3
t5
t6
t7
....
Threads waiting for IO operation response from the OS

```java
public static void main(String[] args) {

    // much higher count for IO tasks
    ExecutorService service = Executors.newFixedThreadPool( nThreads: 100);

    // submit the tasks for execution
    for (int i = 0; i < 100; i++) {
        service.execute(new IOTask());
    }
}

static class IOTask implements Runnable {
    public void run() {
        // some IO operations which will cause thread to block/wait
    }
}
```

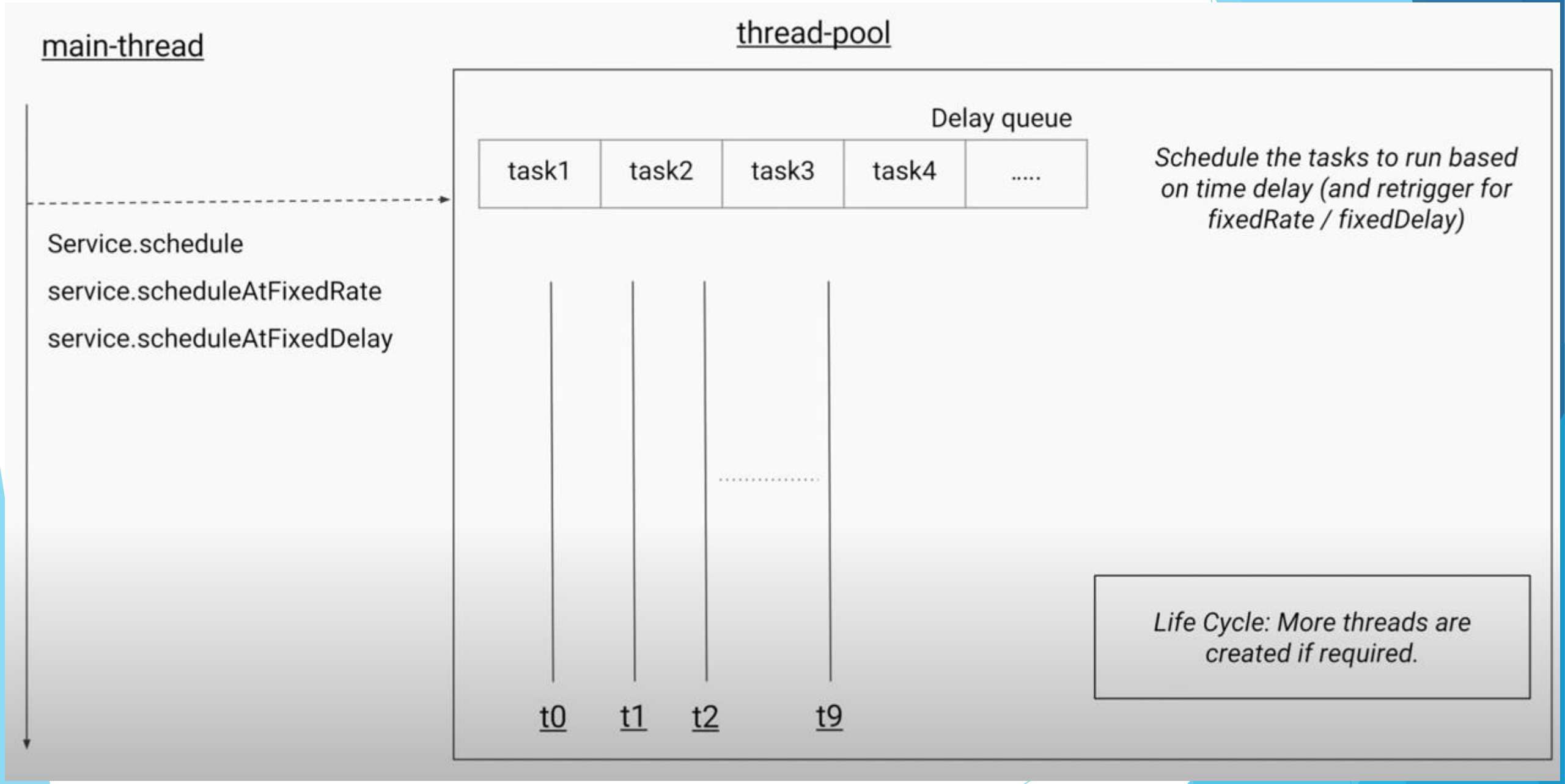
Task Type	Ideal pool size	Considerations
CPU intensive	CPU Core count	How many other applications (or other executors/threads) are running on the same CPU.
IO intensive	High	Exact number will depend on rate of task submissions and average task wait time. Too many threads will increase memory consumption too.

1. FixedThreadPool
2. CachedThreadPool
3. ScheduledThreadPool
4. SingleThreadedExecutor

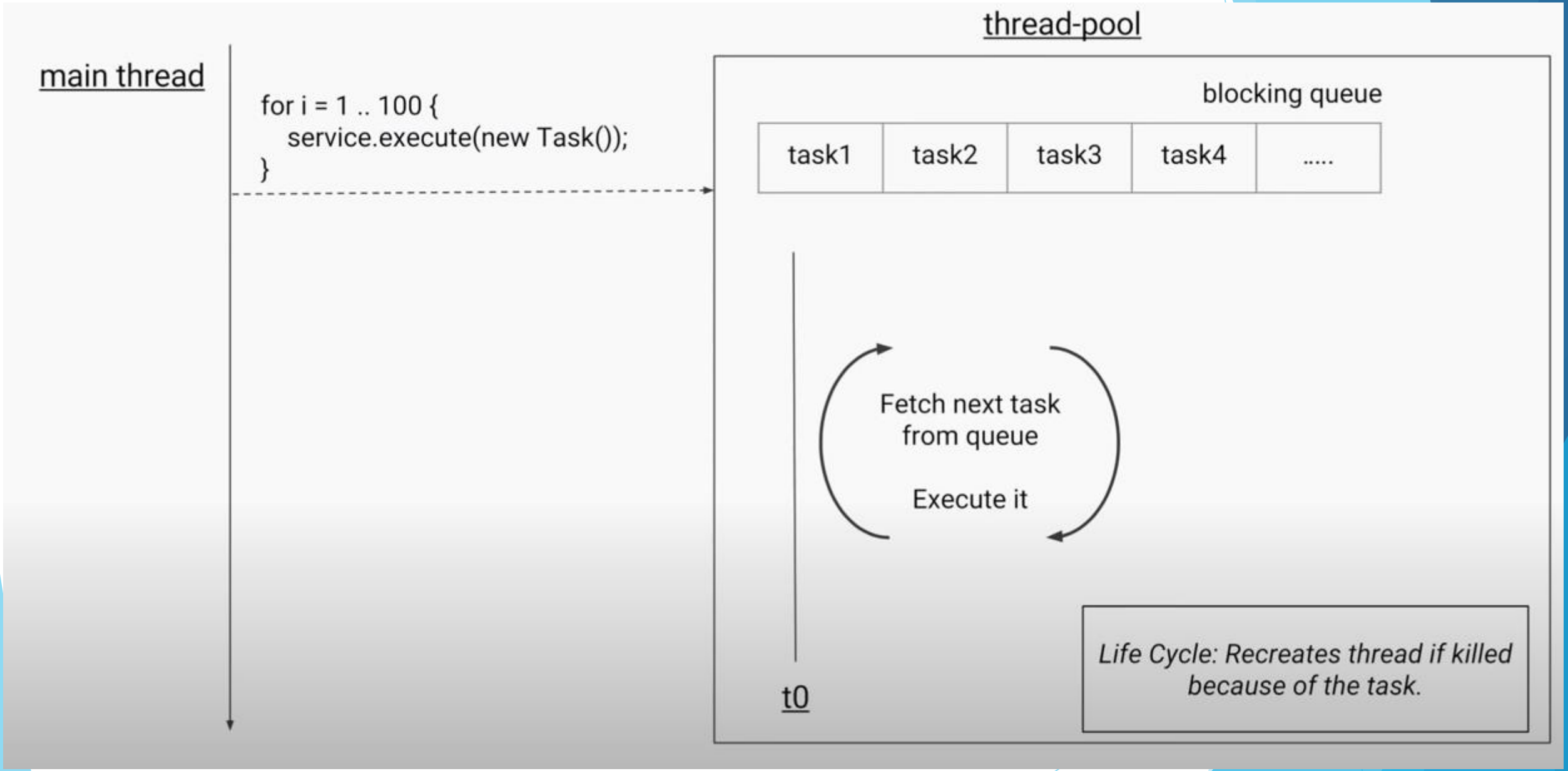

thread-pool
main thread
for i = 1 .. 100 {
service.execute(new Task());
}
blocking queue
task1
task2
task3
task4
.....
Fetch next task
from queue
Execute it
t0
t1
t2
t9


main-thread
thread-pool
Synchronous queue
(can hold only 1 task)
task
for i = 1 .. 100 {
service.execute(new Task());
}
If all threads are busy, then create a new thread for the task and place it in the pool
Life cycle: If thread is idle for 60 seconds (no task to execute) then kill the thread
t0
t1
t2
t9
new

```java
public static void main(String[] args) {

    // for lot of short lived tasks
    ExecutorService service = Executors.newCachedThreadPool();

    // submit the tasks for execution
    for (int i = 0; i < 100; i++) {
        service.execute(new Task());
    }
}

static class Task implements Runnable {
    public void run() {
        // short lived task
    }
}
```


main-thread
thread-pool
Delay queue
task1
task2
task3
task4
.....
Schedule the tasks to run based on time delay (and retrigger for fixedRate / fixedDelay)
Service.schedule
service.scheduleAtFixedRate
service.scheduleAtFixedDelay
t0
t1
t2
t9
Life Cycle: More threads are created if required.

```
blic static void main(String[] args) {

  // for scheduling of tasks
  ScheduledExecutorService service = Executors.newScheduledThreadPool( corePoolSize: 10);

  // task to run after 10 second delay
  service.schedule(new Task(), delay: 10, SECONDS);

  // task to run repeatedly every 10 seconds
  service.scheduleAtFixedRate(new Task(), initialDelay: 15, period: 10, SECONDS);

  // task to run repeatedly 10 seconds after previous task completes
  service.scheduleWithFixedDelay(new Task(), initialDelay: 15, delay: 10, TimeUnit.SECONDS);


atic class Task implements Runnable {
  public void run() {
      // task that needs to run
      // based on schedule
  }
```


thread-pool
main thread
for i = 1 .. 100 {
service.execute(new Task());
}
blocking queue
task1
task2
task3
task4
.....
Fetch next task
from queue
Execute it
t0
Life Cycle: Recreates thread if killed
because of the task.

ForkJoinPool

Semaphore in JAVA